

2. Secure Server Application

The Secure Server Application serves as an overall backend processing and security enforcement layer for the proposed IoT-based environmental monitoring system. Within this role, the Secure Server Application has the responsibility of receiving encrypted MQTT packets originating from the ESP32 sensor node, verifying packet integrity, identifying possible cyberattacks against the server, decrypting sensor payloads, conducting anomaly analysis, and securely storing verified data onto Firebase Firestore.

The Secure Server Application was created using Python and incorporates multiple cybersecurity protective features such as AES-128 decryption, HMAC-SHA-256 verification, prevention of replay attacks, mitigating DDoS attacks, detecting False Data Injection (FDI), detecting malformed packets, and using machine learning-based anomaly detection.

The Secure Server Application establishes a connection to the MQTT broker using a TLS-secured connection on port 8883 and processes all packets being published by the ESP32 sensor node.

2.1 Secure MQTT Communication

The backend server subscribes to the MQTT topic:

'iot/sensor/dht22/secure'

Using the Mosquitto MQTT broker over TLS communication.

- The MQTT receiver is designed to receive encrypted sensor packets and will listen continuously for incoming packets. Security validation occurs as soon as a packet is received.
- Python's SSL/TLS support is enabled for TLS communication during testing; however, certificate verification was disabled temporarily for local implementation and testing.

Table 1

Secure MQTT Communication Components

Component	Description	Implementation in system	Security Purpose
MQTT Broker	Central message broker for IoT communication.	Eclipse Mosquitto Broker	Handles packet routing between ESP32 and backend.
MQTT Topic	Communication channel for secure sensor packets.	'iot/sensor/dht22/secure'	Isolates secure sensor traffic.
TLS Communication	Encrypts MQTT communication channel	'Port 8883 using SSL/TLS'	Protects packets during transmission.
TLS Testing Configuration	Prototype TLS configuration used during development.	'tls_insecure_set(True)'	Enabled encrypted communication during local testing.
MQTT Authentication	Username/password validation for MQTT connection.	'client.username_pw_set()'	Prevents unauthorized broker access.

2.2 AES-128 Payload Encryption and Decryption

Before sending sensor payloads, the ESP32 will encrypt them using AES-128 in CBC mode. When the data reaches the back end, the back end will decrypt it using the same AES encryption key and initialization vector (IV).

Because the plaintext is not guaranteed to be a multiple of the AES block size, we add PKCS7 padding to the plaintext before encrypting it.

Once encrypted, the ciphertext is then encoded with Base64 so that it can be safely transmitted over MQTT.

Table 2

AES-128 Encryption and Decryption Components

Component	Description	Implementation in System	Security Purpose
Encryption Algorithm	Symmetric encryption mechanism	AES-128 CBC	Protects sensor payload confidentiality.
AES Key Length	Shared encryption key size	128-bit key	Provides lightweight secure encryption.
Initialization Vector (IV)	AES CBC initialization value	Static 16-byte IV	Enables CBC encryption mode.
Padding Method	Expands plaintext to block size	PKCS7 Padding	Ensures proper AES block alignment.
Base64 Encoding	Converts ciphertext into text format	<code>mbedtls_base64_encode()</code>	Enables MQTT-safe payload transmission.
AES Decryption	Restores encrypted payload to plaintext	<code>AES.new(...AES.MODE_CBC...)</code>	Allows secure sensor data recovery.
JSON Reconstruction	Converts decrypted text back to JSON	<code>json.loads(plain)</code>	Restores structured sensor data.

2.3 HMAC SHA-256 Integrity Verification

Before sending the encrypted message (ciphertext), the ESP32 generates an HMAC using this ciphertext and a secret key. The backend then generates another HMAC using the received ciphertext and compares it against the transmitted one. If these two values don't match, the data packet is classified as forged/tampered and rejected before decryption.

Table 3

HMAC SHA-256 Integrity Verification Components

Component	Description	Implementation in System	Security Purpose
Hash Algorithm	Integrity verification algorithm.	'HMAC-SHA-256'	Detects packet tampering.
Shared Secret Key	Shared HMAC secret key	'HMAC_KEY'	Ensures trusted verification.
Ciphertext Integrity Protection	HMAC generated using encrypted payload.	'Compute_hmac_hex(ciphertext)'	Protects encrypted packet contents.
HMAC Verification	Compares received and computed hashes.	'expected_hmac == received_hmac'	Detects forged packets
Packet Rejection	Drops invalid packets before decryption	Integrity validation stage	Prevents malicious payload processing.
Security Event Logging	Stores integrity violations in Firestore.	'integrity_violation'	Supports attack monitoring and auditing.

2.4 Replay Attack Detection and Mitigation

Packet Sequence Identifiers are used for detecting replay attacks. In the backend, previously accepted sequence IDs are stored and compared against incoming packets.

When an incoming packet has a sequence ID that is older or the same as an earlier packet, the incoming packet will be considered a replay attack and rejected.

The backend may also track the frequency of attempted replay attacks and will temporarily ban or limit access to devices that have multiple failed attempts of executing a replay attack.

Table 4

Replay Attack Detection and Mitigation Components

Component	Description	Implementation in System	Security Purpose
Sequence Identifier	Unique incrementing packet number.	'sequence_id'	Prevents duplicate packet reuse
Replay Detection Logic	Detects reused sequence IDs.	'seq <= last_sequence_id'	Identifies replay attempts.
Replay Attempt Counter	Tracks repeated replay attempts.	'replay_attempt_times'	Monitors suspicious behavior.
Replay Threshold	Maximum allowed replay attempts.	'REPLAY_THRESHOLD = 5'	Prevents repeated attacks.
Replay Time Window	Monitoring duration for replay attempts.	'REPLAY_TIME_WINDOW = 10'	Enables rate-based replay detection.
Temporary Device Blocking	Blocks devices after threshold exceeded.	'REPLAY_BLOCK_DURATION = 30'	Mitigates continuous replay attacks
Security Logging	Uploads replay events to Firestore.	'replay_attack'	Supports incident auditing.

2.5 DDoS Attack Detection and Mitigation

To detect abnormal behavior from high rates of incoming MQTT packets hitting it, the back-end server continuously checks these packets against a DDoS attack (e.g. volume of packets indicates an attack).

An analysis is then performed over time through a sliding time-window approach for establishing packet rates, after which mitigation controls are enabled if an excessive number of packets are received over a defined threshold condition (configured thresholds).

Table 5

DDoS Attack Detection and Mitigation Components

Component	Description	Implementation in System	Security Purpose
Packet Rate Monitoring	Continuously monitors incoming MQTT packet frequency.	'message_times' list	Detects abnormal traffic spikes.
Detection Threshold	Maximum allowed packet count.	'DDOS_THRESHOLD = 20'	Prevents server flooding.
Time Window Analysis	Monitors traffic within fixed duration.	'DDOS_TIME_WINDOW = 5'	Detects high-frequency attacks.
Sliding Window Mechanism	Removes outdated packet timestamps.	Timestamp filtering	Enables real-time monitoring.
DDoS Detection Logic	Compares packet count against threshold.	'packet_count >= threshold'	Identifies flooding attacks.
Cooldown Blocking	Temporarily ignores incoming packets.	'DDOS_COOLDOWN = 10'	Reduces backend overload.
Packet Rejection	Discards suspicious packets.	Mitigation stage	Protects processing pipeline.
Firebase Logging	Stores attack events in Firestore.	'ddos_attack'	Maintains security records.
Raw Packet Logging	Stores suspicious packet data.	'Raw_packet' field	Supports forensic analysis.
Automated Mitigation	Activates mitigation automatically.	Python backend logic	Enables real-time defense.

2.6 Malformed Packet Detection

The packet structure will be verified before the cryptographic processing can start. Any packets that contain JSON that is invalid or that do not contain required security fields will help to determine if they are suspicious or malformed.

This validation phase prevents incorrectly constructed, incomplete or improperly constructed packets from being decrypted and processed.

Table 6

Malformed Packet Detection Components

Component	Description	Implementation in System	Security Purpose
JSON Validation	Validates MQTT payload format.	'Json.loads(raw_text)'	Detects invalid packet structures.
JSON Decode Error Detection	Detects malformed JSON payloads.	'Json.JSONDecodeError'	Prevents invalid packet processing.
Required Field Validation	Ensures security fields exist.	'sequence_id, ciphertext, hmac'	Detects incomplete packets.
Missing Field Detection	Detects absent cryptographic fields.	'KeyError' handling	Prevents malformed packet attacks.
Packet Rejection	Rejects malformed packets before decryption.	Validation stage	Protects backend resources.
Security Event Logging	Logs malformed packet events.	'malformed_packet'	Supports attack monitoring and auditing.

2.7 False Data Injection (FDI) Detection

False Data Injection (FDI) has several levels of detection in the backend that identify erroneous or not accurate readings from sensors.

The first level of detection is looking for packets that do not have an encrypted packet header type. These are raw readings from the sensor without the expected encrypted headers.

The second level of detection is verifying that the raw temperature and humidity values are in physically permissible ranges before they are analyzed using machine learning.

Table 7 – FDI Detection Components

Component	Description	Implementation in System	Security Purpose
Unencrypted Packet Detection	Detects raw sensor packets without encryption	Missing 'ciphertext' field	Identifies suspicious injected packets.
JSON Structure Validation	Validates packet structure.	'json.loads(raw_text)'	Detects malformed packets.
Temperature Range Validation	Checks acceptable temperature range.	'FDI_MIN_TEMP, FDI_MAX_TEMP'	Detects unrealistic values.
Humidity Range Validation	Checks acceptable humidity range.	'FDI_MIN_HUM, FDI_MAX_HUM'	Prevents invalid sensor injection.
Numeric Validation	Ensures values are numeric.	'float(temperature)'	Prevents malformed sensor data.
Packet Rejection	Rejects invalid injected packets.	FDI mitigation stage	Protects backend integrity.
Firestore Logging	Stores FDI attack details.	'fdi_attack'	Supports attack auditing.

2.8 Machine Learning-Based Anomaly Detection

A trained XGBoost Classifier is used in the backend to detect abnormal environmental behavior after packets pass the appropriate security validation stages. The model performs binary classification by categorizing readings as being either normal or abnormal.

The model was trained with 105,351 labelled records containing IoT sensor data from both indoor and outdoor environments; since the dataset was imbalanced, SMOTE was used during training to improve performance. Additionally, among all evaluated models, XGBoost achieved the best validation F1 score.

Table 8

ML-Based anomaly detection components

Component	Description	Implementation in System	Security Purpose
ML Model	Supervised anomaly detection model	XGBoost classifier	Detects abnormal sensor behavior.
Classification Type	Predicts normal or anomalous readings.	Binary classification	Separates trusted and suspicious readings.
Dataset	Labelled IoT sensor dataset	105,351 observations	Trains the model using real sensor patterns.
Class Balancing	Handles dataset imbalance.	SMOTE oversampling	Improves anomaly detection performance.
Candidate Models	Models evaluated during training.	Logistic Regression, Random Forest, XGBoost	Supports model comparison and selection.
Model Selection	Final selected model.	Highest validation F1 score	Improves precision and recall balance
Normal Class	Represents expected behavior.	Class '0'	Accepts valid sensor readings
Anomaly Class	Represents abnormal behavior.	Class '1'	Flags suspicious readings.
Backend Integration	Runs after security validation stages.	Python receiver inference stage	Ensures only verified packets are analyzed.
Firebase Logging	Stores anomaly detection results.	'anomaly_detected' event	Supports monitoring and alert generation.

2.9 Firebase Firestore Integration

The backend stores validate sensor readings, attack detections, and operational logs within the Firebase Firestore.

Three Firestore collections used in the system are:

1. 'sensor_data'
2. 'Security_events'
3. 'system_logs'

Table 9

Firebase Firestore Integration Components

Collection	Stored Information	Purpose
'sensor_data'	Temperature, humidity, anomaly status, timestamps	Stores validated sensor readings.
'security_events'	Attack detections, mitigation events, packet data	Maintains security event history
'system_logs'	Backend operational logs and status messages	Supports monitoring and troubleshooting.

2.10 System Logging and Monitoring

The backend continuously records operational and security related events during the processing of packets.

These logs support system monitoring, forensic analysis, troubleshooting, and security auditing.

Table 10

System Logging and Monitoring Components

Collection	Description	Purpose
MQTT Connection Logs	Records broker connection status.	Monitors communication stability.
Packet Processing Logs	Tracks packet validation stages.	Supports debugging and tracing.
Security Event Logs	Stores attack detection information.	Enables incident monitoring.
Firebase Upload Logs	Tracks Firestore upload status.	Verifies cloud storage functionality.
ML Prediction Logs	Records anomaly detection results.	Supports ML monitoring.
Mitigation Logs	Records replay and DDoS mitigation actions.	Maintains defense activity history.

2.11 Secure Server Workflow

The backend processes incoming MQTT packets using a multi-stage security validation workflow.

Table 11

Secure server packet processing workflow

Step	Processing Stage	Purpose
1	Receive MQTT packet	Accept incoming sensor packet
2	DDoS detection	Detect packet flooding attacks
3	Packet structure validation	Validate JSON structure
4	HMAC verification	Validate packet Integrity
5	Replay attack detection	Detect duplicated packets.
6	AES decryption	Recover encrypted sensor data.
7	Sequence ID validation	Verify packet consistency.
8	FDI range validation	Detect unrealistic sensor values.
9	ML anomaly detection	Detect abnormal behavior.
10	Firebase upload	Store validated sensor data.
11	Security logging	Record processing events.

Conclusion

The Secure Server Application serves as the primary layer of security and processing in the proposed IoT monitoring system. To accomplish this, the backend will utilize encryption, integrity verification, filtering out malformed packets, preventing replay attacks, preventing DDoS attacks, detecting fraud, applying machine learning analysis, and using Firebase to ensure that only validated and trusted sensor data will be ingested into the system.

The implementation will also show how lightweight IoT environments can be protected from common cyber threats while maintaining secure real-time environmental monitoring and detection of anomalies.